SOURCE-BASED TECHNICAL RESEARCH REPORT

Meta-Harnessing for Manufacturing Systems

Architecture, Agent Engineering, and Decision Support for Industrial Equipment Maintenance

From synthetic ZSK measurements to inspectable operator evidence

This report explains the implemented local application: external time-series classification and retrieval, a saved-model execution loop, persistent context and tasks, permissioned MCP tools, and deterministic completion checks. It distinguishes software verification from industrial diagnosis.

Snapshot	9 October 2026; source and read-only runtime audit
Equipment example	Simulated Coperion ZSK 58 line; synthetic measurements
Audience	AI systems engineers, industrial researchers and operators
Scope	Implementation analysis, evaluation, diagrams and two-page manual

Central principle. The system supports human decisions. It does not authorize physical maintenance, change safety-critical settings or control industrial equipment. Qualified maintenance personnel remain responsible where required.

IMPLEMENTED: verified in code. PARTIAL: implemented mechanisms with material operational or validation gaps. PLANNED: roadmap behavior without current implementation.

Contents

1 The Manufacturing Problem 3

2 Data Modeling and System Requirements 5

3 Time-Series Machine Health Analysis 7

4 RAG-Based Maintenance Knowledge 9

5 Agent-Assisted Maintenance Decisions 11

6 Agent Architecture and Execution Loop 12

7 Memory and Context Management 14

8 Long-Running Tasks and State Persistence 16

9 MCP Tool Connection and Execution 18

10 Security and Authorization 20

11 LLM Connection and Model Integration 22

12 Validation, Verification, and Observability 23

13 End-to-End System Evaluation 24

14 Design Verdicts and Limitations 26

15 Installation and Configuration 27

16 Operating the Decision-Support System 28

References 29

Evidence and reading conventions

Source paths are relative to the repository root unless marked as included report evidence. Figures are original schematics; plots use exact supplied CSV records. Screenshots are genuine repository captures and preserve their original context. Report-time checks, historical recorded tests and illustrative workflows are explicitly distinguished. No secret values are reproduced. The principal unresolved link is asset identity: RAG describes `ZSK-001`, while the synthetic measurements describe `ZSK-SIM-01`. Neither a similar name nor a model label establishes that they are the same physical machine.

PART I — MANUFACTURING PROBLEM AND SYSTEM MODEL

CHAPTER 1

The Manufacturing Problem

A polymer-compounding line converts a variable feed into a controlled melt. Its operator must distinguish a normal change of recipe or throughput from developing mechanical or thermal trouble. An increase in screw torque together with gearbox vibration raises a practical question: what changed, which evidence supports a possible cause, and which inspection should a qualified technician prioritize?

The project's example is a **Coperion ZSK co-rotating twin-screw extruder**. The running RAG catalog records "ZSK Twin-Screw Extrusion Line 01", model **ZSK 58**, manufacturer Coperion GmbH, application Polymer Compounding, and synthetic data provenance. This is a simulated configuration, not evidence that a physical ZSK 58 is connected. The measurement CSV identifies `ZSK-SIM-01`; the RAG metadata identifies `ZSK-001`. No verified asset mapping joins them automatically.

1.1 Components and monitored behavior

The main drive motor supplies mechanical power through the gearbox to two intermeshing screws rotating in the same direction. Screw elements convey, mix and shear material within the barrel. Heating and cooling zones regulate processing temperature. The feeder establishes input mass flow; melt-processing components and the outlet establish pressure and downstream resistance. The software stores one aggregate barrel-temperature signal, not a separately instrumented array of zones.

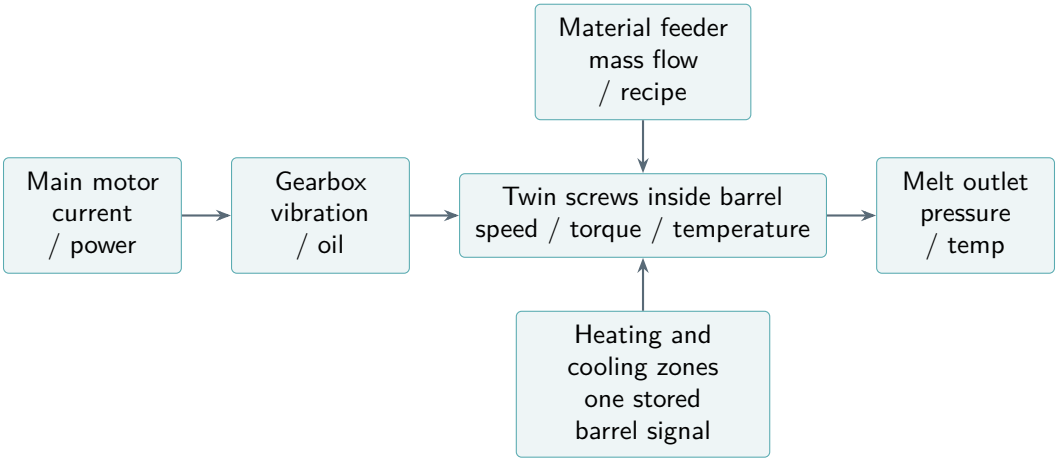


Figure 1.1: Original schematic of the simulated line. Boxes are functional components, not a manufacturer drawing or an installed instrumentation layout.

1.2 The decision is an evidence problem

A vibration rise can be consistent with bearing deterioration, alignment changes or lubrication problems, but RMS vibration alone cannot establish a bearing defect. High torque can reflect process resistance rather than gearbox damage. Recipe, loading, thermal conditions and maintenance history therefore matter. Coperion describes gearbox condition monitoring based on vibrations and oil condition; this motivates the example but does not validate the project's classifier [1].

Engineering verdict. The extruder is a useful integration case because interacting process and mechanical signals require contextual interpretation. The software remains decision support: it neither authorizes physical maintenance nor issues equipment-control commands.

Implementation evidence. `zsk-service/data/zsk_labeled_1000.csv`; `zsk-service/zsk/schemas.py`: `Measurement`; live RAG catalog checked 9 October 2026; `rag-server/rag/models.py`: `Device`.

1.3 Operational concerns and diagnostic scope

Concern	Relevant observation	Interpretation limit
Gearbox vibration	RMS velocity in mm/s; trend with load	No spectral or bearing-frequency measurement
Bearing deterioration	Vibration increase; noise reported by operator	No measured defect frequencies or inspection labels
Oil overheating	Oil temperature, load and vibration	No viscosity, debris or oil chemistry signal
Excessive screw torque	Torque percentage, speed, feed, pressure	Recipe resistance and obstruction can look similar
Barrel instability	Aggregate barrel temperature	No zone-wise heater/cooler diagnosis
Melt-pressure abnormality	Pressure, melt temperature, throughput	No die geometry or material viscosity measurement
Unstable feeding	Feed-rate change with torque variation	No direct hopper/feeder blockage sensor
Excessive motor current	Current, power and torque	No phase currents or electrical fault waveform

These are engineering hypotheses, not validated procedures or machine-specific thresholds. The live RAG issues contain the same eight concern categories, with symptom and possible-cause text. Those records are illustrative maintenance knowledge: they are not verified incidents, work orders or manufacturer fault diagnoses. The database's `possible_cause` field deliberately preserves this distinction.

1.4 Four different questions

Fault detection asks whether observed behavior appears abnormal. The binary synthetic classifier addresses this limited question. **Fault diagnosis** identifies a physical cause and often requires discriminating tests, inspection and domain expertise. **Predictive maintenance** forecasts deterioration or useful life sufficiently to plan intervention. The implementation has no remaining-useful-life model or forecast horizon. **Maintenance decision support** organizes measurements, model outputs, documents and uncertainty for a human decision. These concepts follow the separation between diagnostics and prognostics in condition-based maintenance research [2].

1.5 Requirements derived from the operator's question

The assistant should identify the correct asset, state when each observation was measured, preserve recipe and units, report the actual model version, retrieve relevant passages, and distinguish evidence from interpretation. A recommendation should say which inspection merits consideration and what is still unknown. It should not turn a binary label into an automatic shutdown, repair instruction or certification that operation is safe.

Historical behavior is useful only when comparable operating conditions are identified. A current reading at high load should not be compared indiscriminately with a low-load recipe baseline. The repository does not contain a complete physical asset record, screw configuration, rated motor power, die configuration, approved alarm envelope or plant safety case. Its sensor validation bounds are API input limits, not Coperion operating limits.

Engineering verdict. Combining modalities can improve the explanation available to an operator, but correct diagnosis still requires independently verified asset identity, measurement validity and qualified maintenance judgment.

Implementation evidence. `rag-server/rag/services.py`: issue CRUD and serialization; `zsk-service/zsk/zsk_model.py`: `LABELS`; live issue catalog (eight records).

CHAPTER 2

Data Modeling and System Requirements

2.1 Measured inputs and preserved semantics

The supplied labeled and unlabeled CSVs each contain **exactly 1,000 readings**. They span 1 September 2026 00:00 through 4 September 2026 11:15, with a uniform five-minute interval (83 hours 15 minutes between endpoints). Both contain one machine, `ZSK-SIM-01`, and recipes A/B/C with 350/350/300 rows. The ten signal columns match row for row between the two files; the unlabeled file removes the target and simulated-fault field. These are two representations of the same supplied synthetic history, not independent production datasets.

Column	Unit	SQL representation / meaning
`screw_speed_rpm`	rpm	Double precision; rotation speed
`screw_torque_pct`	%	Double precision; supplied torque percentage
`motor_current_a`	A	Double precision; motor current
`motor_power_kw`	kW	Double precision; motor power
`gearbox_vibration_rms_mm_s`	mm/s	Double precision; RMS vibration velocity
`gearbox_oil_temp_c`	degC	Double precision; oil temperature
`barrel_zone_temp_c`	degC	Double precision; one barrel signal
`melt_temp_c`	degC	Double precision; melt temperature
`melt_pressure_bar`	bar	Double precision; melt pressure
`feed_rate_kg_h`	kg/h	Double precision; mass flow

2.2 Identity, labels and missingness

The baseline uses a bigint row ID, SQL `TIMESTAMP`, text machine ID, constrained recipe and a unique machine/timestamp pair in each source table. Naive supplied timestamps are explicitly interpreted as UTC by `Measurement.utc()`; API additions are converted to UTC before storage. No missing values occur in required CSV columns. `feature_matrix()` rejects non-finite numeric features; no learned imputation pipeline is implemented.

The labeled source has `health_label`: **0=Faulty, 1=Healthy**, and `fault_type_simulated`. There are 847 healthy and 153 faulty rows: 62 gearbox-vibration, 50 flow-obstruction and 41 cooling-issue examples. `alarm_active` is a 0/1 smallint. Fault tags, alarms, timestamps and labels are excluded from model inputs. The fault-generation program is not supplied in this directory; provenance cannot be reconstructed beyond the supplied CSV and its explicit synthetic designation.

Implementation evidence. `zsk-service/migrations/001_baseline.sql`; `zsk-service/zsk/schemas.py`; `zsk-service/zsk/db.py`: `seed()`; CSV audit recorded in `evidence/dataset-audit.json`.

2.3 Current database lifecycle and interfaces

The ZSK service initializes the labeled source once, trains and validates an artifact, then atomically activates its model pointer, removes the training snapshot's labeled rows and imports the 1,000 unlabeled readings. The live database check for this report found **0 labeled rows, 1,000 unlabeled rows and 1,000 separately stored predictions**. Restart never restores the labels because `zsk_state` already exists. Source CSVs remain in the checkout/image; deletion applies to database rows and temporary training snapshots.

`zsk_predictions` preserves source table, measurement ID, model version, predicted label, probability and prediction time. Runtime upgrades add `zsk_state`, `zsk_models`, `zsk_jobs`, `zsk_requests`, `zsk_audit` and job/prediction associations. Model predictions never become ground-truth labels. Separate source identities prevent the same numerical row ID in two measurement tables from being confused.

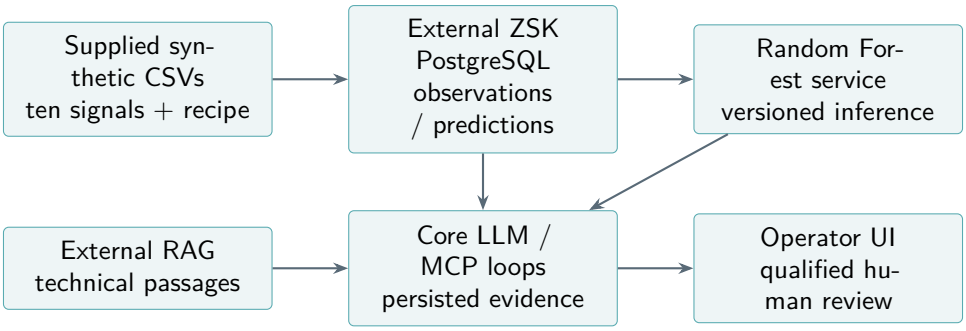


Figure 2.1: Actual data ownership: synthetic measurements and model artifacts remain external to core; the agent receives permitted observations, not direct database access.

2.4 Inputs and outputs across the system

Inputs comprise current and historical readings, recipe/operating conditions, versioned ML results, device/issue metadata, technical documents and the operator's question. "Current" means the most recent available record, not necessarily live telemetry. The supplied history ends in September, whereas the report audit occurs in October; recency must be disclosed.

The desired output contains a health assessment, suspected abnormality, sensor evidence, retrieved sources, possible explanations, inspection priorities and uncertainty. Only the model-result fields have a dedicated ML schema. The six-part maintenance narrative is an authoring recommendation, not an enforced output validator. Ordinary chat stores plain text; durable tasks store an executable decision contract and measurable completion criteria.

2.5 System boundary

UI is React 19/TypeScript/Vite 8/Tailwind 4 with shadcn components. NestJS 12 core runs on Node 24 in Docker and uses parameterized `pg` queries. Core PostgreSQL 17 stores chat/task state. RAG has its own pgvector database, files and local embedding models. ZSK has a dedicated PostgreSQL volume and Random Forest artifacts. No PLC, actuator or physical maintenance controller is connected.

Engineering verdict. The persistence model correctly separates observations, predictions and training targets. A historical synthetic dataset can demonstrate integration, but cannot establish the present condition of a real extruder.

Implementation evidence. `zsk-service/zsk/service.py`: `finish_training()`, `predictions()`; migrations 002/003; `docker-compose.yml`; `core/src/database/database.service.ts`.

PART II — ML, RAG, AND DECISION SUPPORT

CHAPTER 3

Time-Series Machine Health Analysis

3.1 Problem and selected approach

A single-measurement classifier considers only one instantaneous signal vector. A time-window classifier supplements it with recent behavior. Time-series anomaly detection can learn deviation from expected dynamics without equivalent supervised fault labels. Predictive-maintenance forecasting estimates future condition or time to failure. This service implements **supervised binary classification with causal short-window features**; it is neither a sequence model nor a prognostic forecaster.

`feature_matrix()` constructs **20 features**: ten measured signals, two recipe indicators (B and C, with A as reference), and mean/standard-deviation pairs for torque, vibration, oil temperature and melt pressure. A per-machine deque retains at most three readings, including the target. At five-minute cadence a full window spans ten minutes from first to last, not fifteen minutes of elapsed history. Population standard deviation uses NumPy's default `ddof=0`; first/second rows use shorter warmup windows.

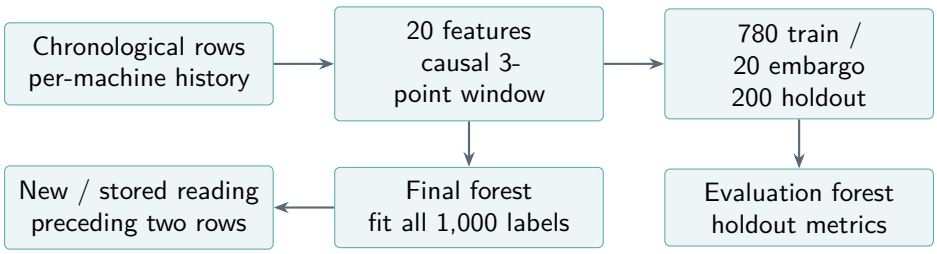


Figure 3.1: Actual feature and model pipeline. The evaluation candidate and final all-labeled model are distinct artifacts in the training procedure.

3.2 Design decision and training procedure

Random Forest provides nonlinear interactions between load, thermal and vibration features without a large sequence-training stack. This is a reasonable proof-of-concept choice, not evidence of an algorithm comparison. `_classifier()` fixes 180 trees, maximum depth 9, minimum leaf size 4, balanced class weights, seed 20261009 and parallel CPU execution. The repository has no hyperparameter search or industrial calibration study. Random Forest probabilities aggregate tree estimates; they are not validated failure-risk probabilities [3].

`train()` sorts by time/machine, builds causal features, requires at least 100 labeled rows and both classes in train/test partitions, evaluates an earlier-period candidate, then trains the deployed forest on all labeled rows. Random Forest has no `partial_fit()` here: new verified labels trigger **full retraining**, not incremental fine-tuning. The service rejects training when no labels remain.

Implementation evidence. `zsk-service/zsk/zsk_model.py`: `MEASURED`, `ROLLING`, `FEATURES`, `feature_matrix()`, `_classifier()`, `train()`.

3.3 Measured history and model lifecycle

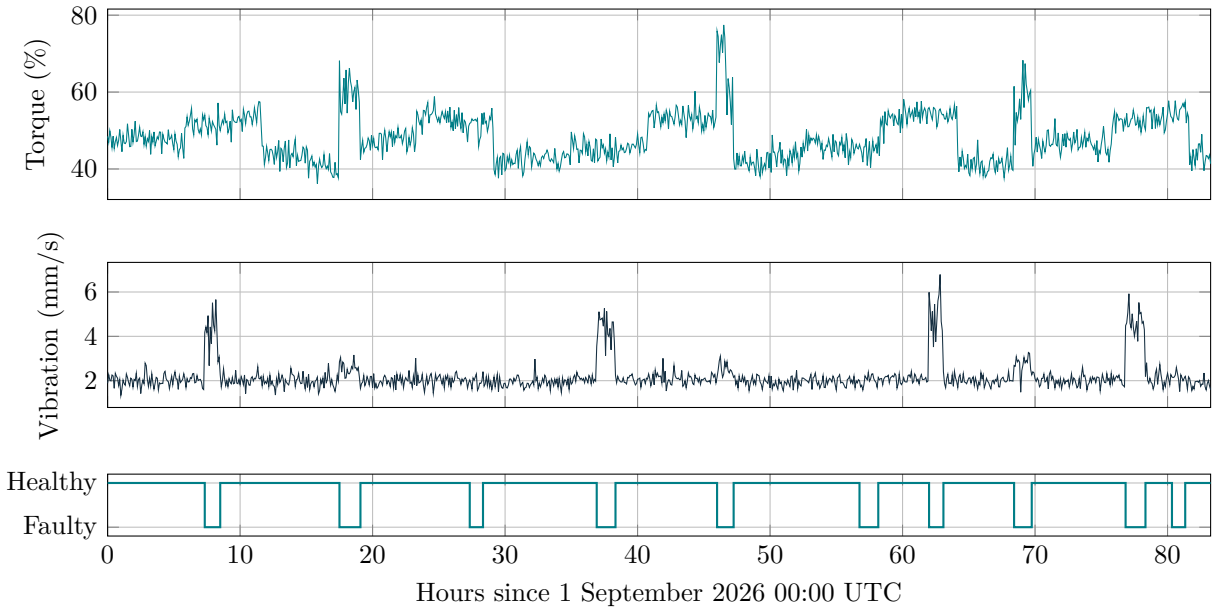


Figure 3.2: Exact supplied synthetic history (all 1,000 rows). The bottom strip shows actual supplied targets, not predicted health. No OEM alarm thresholds are drawn.

`Service.predict()` resolves either a stored ID or a supplied reading and obtains the previous two same-machine observations. It checks machine/time order; new readings need two prior rows. Stored early rows may use warmup windows and report that fact. `batch_job()` freezes target membership and model version, retrieves history outside each page/date boundary, predicts a combined chronological window and stores only target outputs. This prevents artificial probability changes at pagination boundaries.

`add()` validates exact units, finite bounded fields, source/provenance and a stable request key. Per-machine/source advisory locks enforce append-only timestamp ordering. Duplicate or out-of-order readings are rejected per row. Labels require `verified_operator_label` provenance; this is an operator assertion, not independent truth verification. Predictions remain in their own table.

`train_job()` writes a temporary CSV/artifact, smoke-tests inference, fsyncs, atomically renames the artifact and registers a SHA-256 checksum. The training job UUID is the model version. `artifact()` loads only registered, service-owned non-symlink artifacts with matching hashes; `loaded()` checks feature/class compatibility. Activation/purge commits in SQL; rollback selects a validated registered version. A failed transaction may leave an unregistered artifact, which cannot be activated by an arbitrary upload.

Engineering verdict. Causal feature construction and preserved page-boundary history make inference internally consistent. Five-minute RMS signals and three-point windows cannot establish high-frequency mechanical faults or future useful life; synthetic patterns limit external validity.

Implementation evidence. `zsk-service/zsk/service.py`: `history()`, `predict()`, `batch_job()`, `add()`, `train_job()`, `artifact()`, `rollback()`.

CHAPTER 4

RAG-Based Maintenance Knowledge

4.1 Problem and external knowledge model

"Faulty" is a classification, not a causal explanation. The external RAG service supplies passages and illustrative known-issue descriptions for interpretation. It is an independent FastAPI/Python MCP SDK application with an ingestion worker and PostgreSQL/pgvector, and contains **no generative LLM or answer engine**. Retrieval-augmented generation combines retrieved external information with model reasoning; this application performs that composition in core rather than jointly training a retriever/generator [4].

Domain table	Relationship and purpose
`devices`	Exact name/model/manufacturer identity plus metadata
`issues`	Device FK, symptoms, category, possible cause and embedding
`documents`	Stored file, version, ingestion status and provenance metadata
`device_documents`	Many-to-many assignment of documents to devices
`document_chunks`	Passage, embedding, section/page/order and document FK
`issue_evidence`	Explicit issue/chunk link with evidence type

Device–issue–evidence connects a curated concern to a particular passage. Device–document–chunk also enables direct retrieval when no known issue has been curated. Evidence types are possible cause, diagnostic procedure, resolution and supporting observation. Similarity never creates these links automatically. The live catalog currently has **zero explicit issue-evidence links**; direct document retrieval is therefore particularly important.

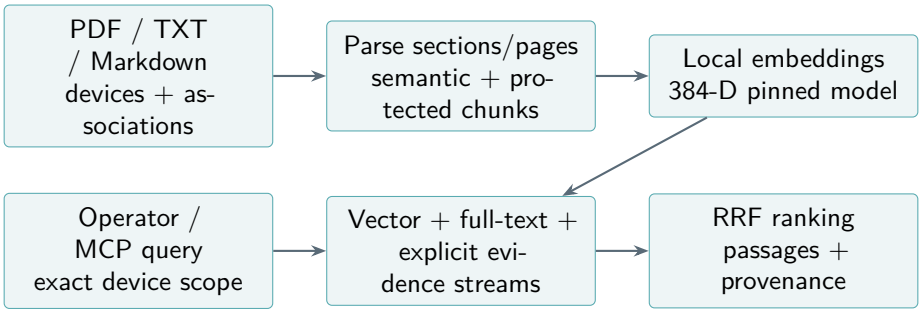


Figure 4.1: Independent ingestion and hybrid retrieval. Explicit evidence links are supported in code; none existed in the audited live ZSK catalog.

4.2 Ingestion implementation

`DocumentService.upload()` validates identity/associations, stores source size/hash and safe local filename, and handles duplicate/concurrent uploads. The worker claims a document through a database advisory lock. PDF text comes from pypdf; UTF-8 TXT/Markdown is supported. Scanned documents require external OCR. Structural headings/pages are preserved before semantic splitting; warning/procedure blocks receive protected handling. Semantic splitting uses LlamaIndex inside sections, with bounded token fragments when required.

Implementation evidence. `rag-server/rag/models.py`; migration `0001_knowledge.py`; `rag-server/rag/documents.py`: `DocumentService`; `rag-server/rag/worker.py`: `IngestionWorker.process()`; `rag-server/rag/chunking.py`: `extract()`, `structures()`, `Chunker.split()`.

4.3 Retrieval, attribution and actual document corpus

`LocalEmbeddings` uses pinned `all-MiniLM-L6-v2`, 384-dimensional normalized local vectors, CPU by default and a 220-token chunk bound. Model revision/dimension/normalization form a persisted embedding contract. Queries use the model's query-embedding method. No OpenAI embedding key is required; initial model acquisition can require network access.

`RetrievalService.search()` resolves the device deterministically, rejecting conflicting/ambiguous filters. It searches issues by cosine similarity and PostgreSQL English full-text ranking, and retrieves ready document chunks through four streams: direct vector, direct keyword, explicit issue evidence, and documents associated with matched issues' devices. Reciprocal-rank fusion uses

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{60 + \mathrm{rank}_r(d)}.$$

Defaults are 50 candidates, passage similarity threshold 0.2, issue threshold 0.3 and top 10 passages. Optional pinned local cross-encoder reranking exists but is disabled by default. Scores express ranking, not diagnostic confidence.

Live document title	Chunks	Source role
ZSK Twin-Screw Extruder Brochure	160	Official Coperion 2025-08 product publication
ZSK Condition Monitoring Extruder Brochure	17	Local title for a September 2025 ZSK/STS press release
Extruder Modernization and Service	26	Official service/modernization brochure

The catalog's "Condition Monitoring" title does not turn a press release into a diagnostic manual. Original URL, filename/hash, document UUID/version, page number, heading/path, chunk UUID/order and exact text must travel with evidence. PDF extraction in the main brochure sometimes yields spaced words and misleading headings; a page reference can remain useful even when the inferred heading is poor.

4.4 Read-only check executed for this report

A real REST search scoped to the ZSK 58 catalog entry returned three passages for "gearbox vibration oil condition monitoring maintenance". The top passage was the September 2025 publication, page 3, chunk `a6d1ed64-14f7-46fd-b54f-7172a978f476`; another came from the main brochure, page 28. The full response is in `evidence/rag-readonly-search.json`. This verifies retrieval on existing data, not an agent's successful maintenance diagnosis. The source publications discuss condition monitoring and service support; they do not supply validated thresholds for this synthetic dataset [5, 6, 7].

Engineering verdict. Hybrid retrieval and explicit source identities make external knowledge inspectable. Sparse curated links, extraction artifacts and brochure-level evidence limit diagnostic specificity. An ingested document's official provenance does not make every retrieved interpretation correct.

Implementation evidence. `rag-server/rag/retrieval.py`: `resolve_device()`, `document_scope()`, `search()`, `rrf()`; `rag-server/rag/embeddings.py`; `rag-server/rag/config.py`.

CHAPTER 5

Agent-Assisted Maintenance Decisions

5.1 Actual mechanisms and intended composition

An operator can ask the saved model to inspect measurements, invoke prediction, find a device, retrieve issues and search documentation. Tools are generic; there is no hardcoded ZSK diagnostic pipeline. The following workflow is **intended composition of implemented capabilities**, not a recorded complete ZSK maintenance run.

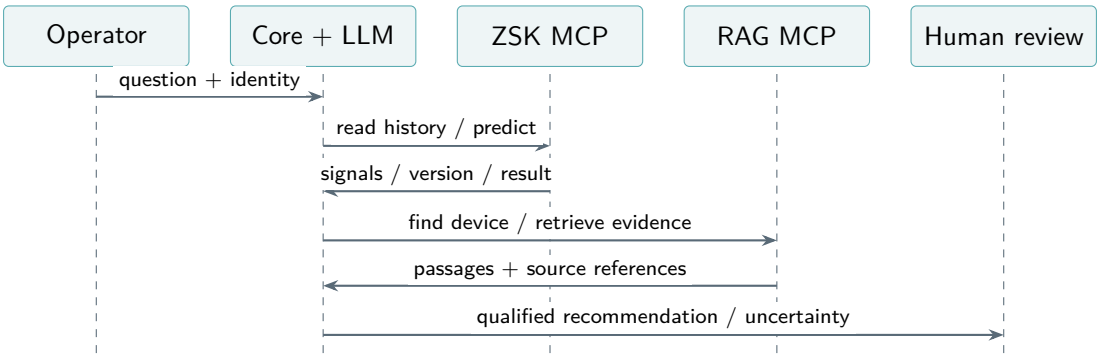


Figure 5.1: Illustrative end-to-end sequence using implemented tools. Asset mapping and industrial validation remain human responsibilities; the complete sequence was not executed as one agent run.

1. Receive the question; identify exact dataset machine and RAG device. Request an explicit mapping when names disagree.
2. Read bounded current/history pages with timestamps, recipe, units and provenance; check freshness before describing a present state.
3. Invoke `predict_zsk_health` for a stored row; inspect history count, warmup flag and active version. Read actual versioned metrics when relevant.
4. Resolve the equipment through `find_device`, read `get_device_issues`, retrieve `search_knowledge` and obtain exact passages with `get_document_context` if needed.
5. Compare signal trends, model result and technical passages. Run available execution checks and expose missing industrial checks.
6. Produce an evidence-qualified recommendation for the operator and technician, keeping hypotheses separate from measurements.

5.2 Worked example: recorded result plus limited interpretation

`docs/verification/zsk-agent.json` records successful calls to `predict_zsk_health` (177 ms) and `get_zsk_model_metrics` (38 ms). Measurement 1000, time 4 September 2026 11:15, was **Healthy (1)**, fault probability **0.0**, with two preceding readings and no warmup. Its model UUID begins `5d27c404`. The underlying CSV has torque 41.36%, vibration 1.593 mm/s and oil temperature 43.58 degC. A zero model probability is not zero physical risk.

A defensible illustrative answer separates **observation** (those historical synthetic values), **model result** (label/probability/version), **evidence** (the independently retrieved publication and page), **interpretation** (no confirmed mechanical cause), **recommendation** (verify current readings and asset identity before choosing an inspection), and **uncertainty** (no current telemetry, real labels or approved procedure). This format is not enforced by core. The recorded agent reply did not include the later RAG check or authorize maintenance.

Engineering verdict. The integration demonstrably delivers real ML results to the saved LLM. Combining them into a correct industrial recommendation remains partial and unvalidated, especially across the two asset identifiers.

PART III — META-HARNESS ENGINEERING

CHAPTER 6

Agent Architecture and Execution Loop

6.1 Problem, terminology and design

An LLM proposes text or a tool action. An agent couples these proposals to an action–observation loop. A harness supplies execution policy, context construction, persistence, limits and verification. This project's name "Meta-Harness" denotes its local control plane around a saved LLM and external services. It does **not** integrate multiple external coding-agent runtimes. In Barbaste et al.'s narrower terminology, a meta-harness orchestrates harnesses from above; this project instead implements its own agent loop. The paper's separation of loop, model integration, tools, memory and safety is useful as an analysis framework, not a claim of equivalent architecture or benchmark performance [8].

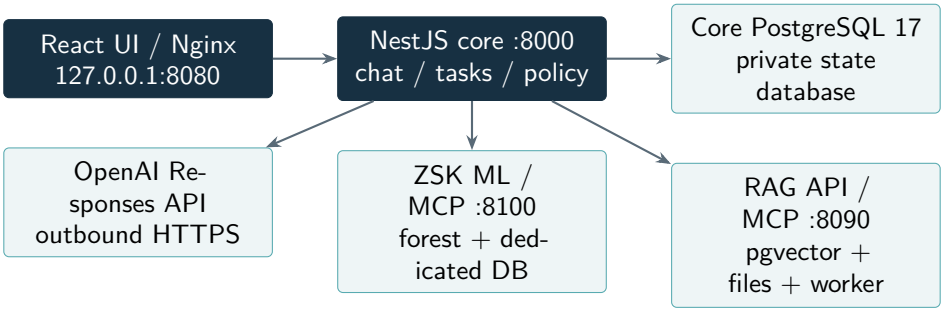


Figure 6.1: Complete implemented topology. Local host bindings coexist with outbound OpenAI inference; selected tool observations can leave the machine in model context.

6.2 Two loops share one LLM service

`ChatsService.respond()` serializes a conversation with a session advisory lock, selects ordered history, reconstructs private context and calls `LlmService.respond()`. OpenAI Responses emits function calls or final text. The service validates each call, asks for exact approval for writes, executes sequentially and appends `function_call_output` observations. Final text is persisted once per user message.

Durable tasks use `TaskWorker.run()` and `LlmService.taskDecision()`. The model receives a bounded snapshot and proposes a strict decision with goal, plan, criteria, step, action and summary. The worker interprets execute/wait/complete/blocked/respond, controls authority and persists results. There is no second reasoning model, queue microservice or agent database.

Engineering verdict. Owning the loop in core keeps side effects under explicit policy and persistence control. The term Meta-Harness is a project label; reliability comes from these concrete mechanisms rather than from a model or runtime brand.

Implementation evidence. `core/src/chats/chats.service.ts`: `respond()`; `core/src/llm/llm.service.ts`: `respond()`, `taskDecision()`; `core/src/tasks/task-worker.ts`: `run()`; `core/src/app.module.ts`.

6.3 Decision generation, execution and stopping

The conceptual sequence is propose–validate–act–observe–verify–continue. It does not expose or validate a model's hidden "thinking". In chat, a text-only completed response ends the loop without deterministic industrial verification. In tasks, a `complete` proposal must pass stored evidence predicates before the task becomes COMPLETED.

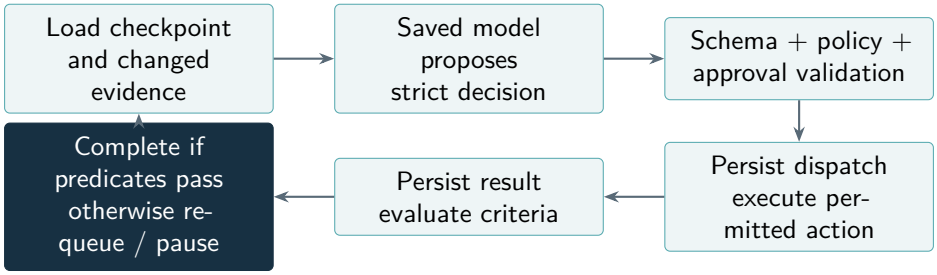


Figure 6.2: Durable decision loop. Approval and uncertainty are persisted pauses; completion is a backend decision based on evidence predicates.

Listing 6.1: Source-aligned worker pseudocode (abbreviated).

```
claim due task; acquire session lock and lease owner
load checkpoint, next_action and operation ledger
if prompt budget reached: summarize, checkpoint, requeue
if no saved decision:
  build context delta; ask saved LLM; parse strict decision
  atomically save decision, context and delivery cursor
if complete: verify stored criteria; finish only if passed
if wait/respond/blocked: handle explicit branch
if execute:
  validate policy; reuse any completed identical write
  persist operation; obtain exact approval or bound grant
  persist DISPATCHING immediately before external call
  persist observation; verify; advance and requeue
on interruption: retry reads; pause uncertain writes
```

6.4 Malformed decisions and finite execution

`parseTaskDecision()` extracts one unambiguous JSON decision without evaluating code or repairing syntax. Ajv checks the schema; `validateDecision()` checks action semantics and measurable criteria. Responses are bounded to 262,144 characters; extraction examines at most 512 objects. Conflicting decisions, invalid argument JSON or absent actions cause a transient planner error before execution. The first usable plan's criteria persist rather than being silently weakened by later proposals.

Chat limits are eight model rounds, twelve tool calls and 120 seconds per prompt. Checkpoints renew the prompt; UI stops automatic continuation after 20. Three deadline prompts without progress expose retry. Durable defaults are eight decisions, 200,000 accounted tokens and 120,000 active ms per prompt, with 20 continuations; slice deadline is also 120 seconds. Lifetime token/time/cost accounting persists. These are finite safeguards, not a semantic proof of progress or absence of repetitive reasoning.

Implementation evidence. `core/src/tasks/task-decision-parser.ts`; `core/src/tasks/task-contract.ts`: `validateDecision()`; `core/src/tasks/task-prompt.ts`; `ui/src/App.tsx`: `generateResponse()`; `core/src/tasks/task-worker.ts`: `continuePrompt()`.

CHAPTER 7

Memory and Context Management

7.1 Problem: continuity without conflating memory and state

Conversation memory is the visible record of user/assistant exchanges. Working context is the actual private input window used for reasoning, including prior tool results and provider compaction items. Persistent execution state records authoritative actions, approvals and checkpoints. External RAG knowledge is curated technical content retrieved when needed. These stores answer different questions and must not substitute for each other.

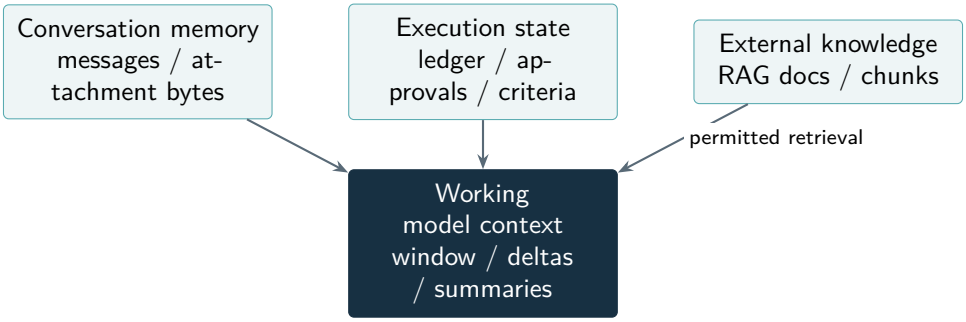


Figure 7.1: Four kinds of continuity. Summarizing model context never deletes the visible transcript or authoritative operation ledger.

7.2 Implemented conversation reconstruction

`chats` stores title/activity and a private `context_window`. `messages` stores ordered role/text, reply linkage, compact traces and private `tool_state`; `attachments` stores metadata and bytes. Message ordering uses the persisted position. A unique reply linkage makes a retry return an existing answer rather than create another final reply.

The private window contains model ID, full returned context items and a `throughPosition` watermark. `ChatsService.respond()` appends only messages beyond that watermark for the same model. A model switch reconstructs from the visible transcript. Completed tool results and pending calls live in continuation state, allowing provider retries without automatically repeating a dispatched write. Restart loads these SQL fields instead of reconstructing work from a natural-language summary alone.

7.3 Input assembly and attachment boundary

Core passes chat instructions, the selected model/thinking settings, available allowed-tool schemas and serialized conversation to the provider. Tools are discovered dynamically; ordinary chat does not force a retrieval call. File bytes remain in PostgreSQL for download. The model receives filenames with an explicit content-unavailable note; attaching a manual to chat is **not** document ingestion. RAG Documents is the supported route for searchable passages.

Engineering verdict. Separating model memory from state makes continuity auditable and protects the transcript during compaction. It does not guarantee that every relevant past fact will survive model summarization or a change of selected model.

Implementation evidence. `database/schema.sql`; `core/src/chats/chats.service.ts`: `get()`, `respond()`; `core/src/llm/context-window.ts`: `ChatContextWindow`; `core/src/llm/llm.service.ts`: `respond()`.

7.4 Budgeting, compaction and task evidence delivery

7.4.1 Exact provider counting before inference

`countContext()` requests `/v1/responses/input_tokens`. At 90% of the configured capacity, `compactContext()` calls the selected model's `/compact` endpoint. Core retains the *entire* returned canonical window, including opaque compaction/reasoning items, checks the 16 MiB serialization bound and recounts. If the new window remains above threshold or compaction is unsupported, inference fails visibly while saved history/evidence remains available.

`contextBudget()` permits an operator override of 8,192–2,000,000 tokens. Code contains capacity entries for three named models and a 128,000-token fallback; these are implementation configuration, not independently established provider guarantees. Output is bounded by the smaller of 8,192 tokens and 8% of capacity. Listing a model does not prove that counting, compaction or thinking is compatible.

7.4.2 Incremental task context

`taskContextDelta()` fingerprints operation status/result/error/attempts. Initial context includes request, plan/criteria and up to 20 conversation messages, each limited to 4,000 characters. Subsequent turns receive changed observations and changed tool metadata plus the latest 40-operation ledger, ten completed-step labels and deterministic check statuses. Results above 12,000 characters become marked previews. Updates target a 64,000-character changed-evidence batch; a single first oversized change can exceed that target, so it is not a universal hard cap.

Delivery fingerprints and the private window commit together under owner fencing. `readTaskEvidence()` reads only this task's stored operation IDs and applies JSON-pointer/offset/limit paging. Arrays are limited to 50 items and reduced by character size; an oversized single item becomes a preview and needs a more specific pointer. Paging reads stored facts; it never re-executes a remote operation or counts as new external proof.

7.4.3 Long-task checkpoints and pollution control

At a per-prompt turn/token/time boundary, `summarizeTaskContext()` requests a factual memo from the same model (2,048 output tokens, 16,000-character returned-text limit). `continuePrompt()` commits the memo, resets prompt baselines, retains lifetime accounting and requeues. Failed summaries do not overwrite saved context. Immutable criteria, grant identities and write ledger survive independently of prose.

Instructions label server descriptions and retrieved content untrusted and preserve hypotheses, IDs and declined/uncertain operations. This mitigates prompt pollution but does not eliminate injection. Stale observations can survive a summary; industrial freshness must be checked explicitly. There is no semantic chat-memory index or learned personal-memory subsystem.

Engineering verdict. Bounded deltas and stored-result paging reduce repeated context growth while preserving exact evidence access. Summaries remain lossy, and the current implementation does not deterministically enforce observation freshness.

Implementation evidence. `core/src/tasks/task-context.ts`: `taskContextDelta()`, `readTaskEvidence()`; `core/src/llm/llm.service.ts`: `summarizeTaskContext()`; `core/src/llm/context-window.ts`.

CHAPTER 8

Long-Running Tasks and State Persistence

8.1 Problem and persistence design

A multi-step investigation can outlast a browser request or backend process. Core therefore stores tasks, ordered decisions, external-operation results, approvals, checks, events and schedules in its existing PostgreSQL database. `agent_tasks` includes request/goal/plan/criteria, current/completed steps, next decision, status, progress, retry timing, private context, delivery cursor, prompt baselines, lifetime usage and lease ownership.

`TaskWorker` polls each second. Default concurrency is two active slices, configurable from one to four. `claim()` selects due PENDING/RETRYING tasks or expired RUNNING leases using `FOR UPDATE SKIP LOCKED`, obtains a session advisory lock, assigns a fresh owner UUID and commits a 30-second lease. A five-second heartbeat extends it. `fenced()` locks the task row and checks RUNNING plus the same owner before committing the next planning/checkpoint transaction.

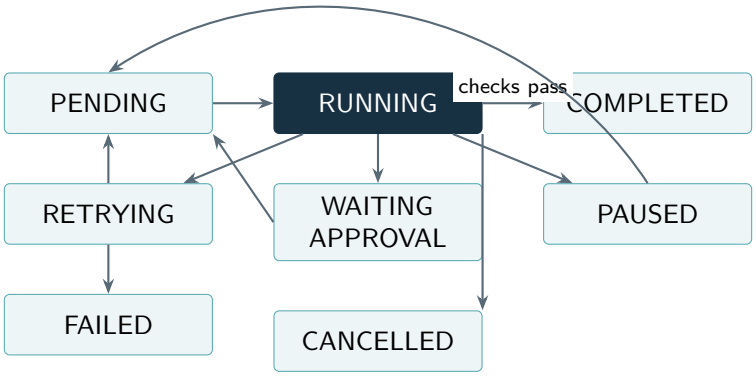


Figure 8.1: Persisted task lifecycle. UNKNOWN is an operation state that forces a task to PAUSED; it is not a ninth task status.

8.2 Recovery distinguishes reads from writes

Before dispatch the worker saves a proposal/operation, then commits DISPATCHING immediately before the external request. On recovery, a dispatched read becomes FAILED and can safely retry; a dispatched write becomes UNKNOWN, and the task pauses. A completed write is reused by canonical action identity or stable plan-step key. Core sends operation UUID in `Idempotency-Key` and MCP `_meta.operationId`; the server may ignore those fields.

Cancellation or pause revokes the owner's ability to advance. Heartbeat detects loss and aborts local execution, but an already sent remote action cannot be rolled back. Result-recording transactions intentionally retain a returned operation outcome; they are not a distributed fence on the remote service. A database partition can still leave ambiguous external effects.

Engineering verdict. PostgreSQL checkpoints, ownership checks and conservative UNKNOWN handling support restart recovery. They provide replay protection, not exactly-once execution across unrelated services.

Implementation evidence. `core/src/tasks/task-worker.ts`: `claim()`, `fenced()`, `run()`, `advance()`; migrations 001–003; `core/src/tasks/tasks.service.ts`: `control()`.

8.3 Operation ledger, retries and reconciliation

Operation state	Recovery meaning
PREPARED	Proposal persisted; no dispatch recorded
DISPATCHING	External send may have occurred
SUCCEEDED	Actual result persisted; a completed write is reused
FAILED	Failure recorded; replay depends on policy and outcome
UNKNOWN	Write outcome unresolved; inspect remote state first
REJECTED	Operator declined; revise rather than repeat

`retryDelay()` uses exponential backoff and jitter, starting at one second, with a five-minute base cap and Retry-After support. The default allows five retries after the initial failure. A read-only tool rejection retains the failed result and asks for a corrected decision; unsafe policy/authentication or permanent failures stop. Network unavailability can trigger rediscovery only for enabled, permitted profiles not manually disconnected.

`TasksService.reconcile()` records an operator's externally checked outcome: succeeded or definitely not executed. It does not promote supplied narrative/result data into completion evidence. `checkCriterion()` excludes operator-reconciled results, requiring a fresh verification read. Terminal COMPLETED/CANCELLED tasks cannot be resumed; FAILED uses Retry, and PAUSED uses Resume once pending approvals/unknown writes are resolved.

8.4 Schedules and schema evolution

`TaskSchedulesService.save()` validates one-time or cron scheduling and IANA timezone. `tick()` transactionally inserts a task using a schedule/time dedupe key, then advances or disables the schedule. Missed cron occurrences coalesce into one catch-up task; future recurrence is computed from current time. Invalid task templates disable with a persisted explanation rather than starving the queue. Scheduled requests keep configured scope; the schedule form supplies no broad write grant.

Event-driven configuration, webhook intake and trigger execution are temporarily removed. Historical trigger tables remain inert in immutable migrations. Earlier logs mention event-authentication tests; they are historical evidence, not available product behavior.

`DatabaseService.onModuleInit()` applies the schema and lexically ordered migrations under a transaction/advisory lock. Recorded SHA-256 checksums reject changes to applied upgrades. Migration 001 adds durable execution, 002 cost accounting and 003 context checkpoints. This is a reproducible upgrade mechanism, not a promise of painless arbitrary schema changes.

8.5 Recorded recovery test

The stored task restart fixture killed a container during a read after a committed write. The task completed with one write and two read attempts. It used real PostgreSQL/worker/MCP and a deterministic planner, not a paid live model. ZSK separately records a killed batch resuming to 1,000 outputs with attempts=2. Those results are finite examples, not adversarial partition/concurrency proofs.

Implementation evidence. `core/src/tasks/tasks.service.ts`: `reconcile()`, `complete()`; `core/src/tasks/task-schedules.service.ts`; `docs/verification/task-restart.json`; `docs/verification/zsk-restart.log`.

CHAPTER 9

MCP Tool Connection and Execution

9.1 Problem and generic tool abstraction

MCP connects core to external services without embedding their business logic. Core is the client/host; independent ZSK and RAG applications are servers. Server discovery supplies tool names, descriptions, JSON schemas and annotations; the registry stores these with endpoints, status and explicit operator allowlists. Streamable HTTP is the only implemented MCP transport. MCP standardizes discovery/calling, while input validation, authority, network controls and evidence handling remain host responsibilities [9].

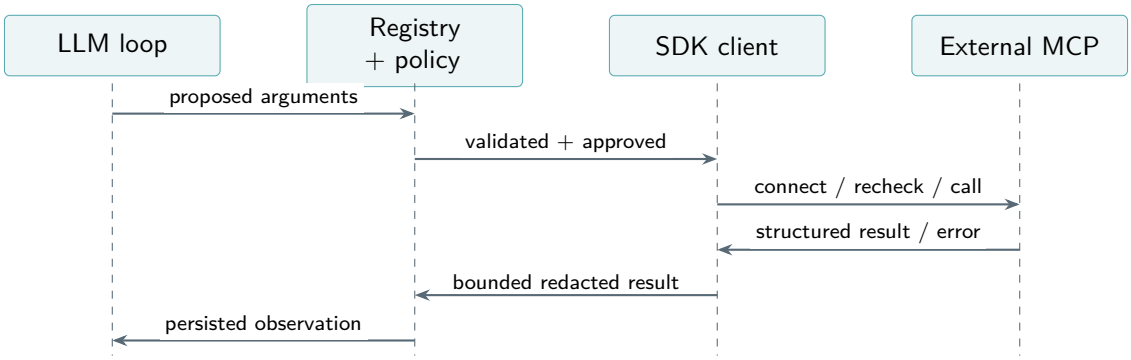


Figure 9.1: Actual call ordering in McpService. Policy is rechecked after connection and before dispatch; observation persistence occurs in the owning chat/task loop.

9.2 Registry, names and bounded discovery

`McpService.inspect()` initializes the official TypeScript SDK client and lists tools across cursor pages. Discovery has a 15-second overall abort and 10-second list timeout; metadata is bounded to 500 tools/2 MiB, tool schema to 64 KiB, names to 200 characters and descriptions to 2,000. Client/transport close in `finally`. Bootstrap reconnects enabled agent profiles; save/test/disconnect states are persisted.

`agentTools()` exposes up to 64 currently connected, enabled, allowlisted tools. Provider-facing names hash server UUID plus tool name, preventing ordinary cross-server name collisions while preserving the original identity for dispatch. Schema metadata is forwarded with `strict:false`; the generic interface does not claim every remote schema fits OpenAI strict-tool requirements.

9.3 Validation and execution

`prepareCall()` rechecks profile status/permissions, requires an object of at most 64 KiB and validates it with Ajv. Regex `pattern`/`patternProperties` schemas are intentionally rejected; formats are not validated there. External Pydantic validation remains necessary. `execute()` rechecks configuration digests after connecting, then invokes the SDK. Call timeout is 15 seconds, overall execution timer 20 seconds, serialized result bound 256 KiB and network response bound 4 MiB. MCP `isError` prevents a successful-result claim.

Engineering verdict. MCP supports independent service evolution and generic orchestration. Bounded schemas and permissions are necessary, but a registered server's metadata and returned content still form a trust boundary.

Implementation evidence. `core/src/mcp/mcp.service.ts`: `inspect()`, `agentTools()`, `prepareCall()`, `execute()`; `core/src/mcp/network-policy.ts`; `zsk-service/zsk/mcp_server.py`; `rag-server/rag/mcp_server.py`.

9.4 Manufacturing and RAG tools in practice

Tool	Access	Implemented response/action
`get_zsk_measurements`	Read	Paginated signals and units
`predict_zsk_health`	Read	Label, probability, history and version
`get_zsk_model_metrics`	Read	Versioned evaluation metrics
`get_zsk_training_status`	Read	Durable training/batch status
`get_zsk_predictions`	Read	Stored output pages
`add_zsk_measurements`	Admin write	Validated append with stable key
`train_zsk_health_model`	Admin write	Queue full retraining
`batch_predict_zsk_health`	Write hint	Queue/store inference; status branch
`find_device`	Read	Exact matches/ambiguity
`get_device_issues`	Read	Device-scoped illustrative concerns
`search_knowledge`	Read	Hybrid ranked passages/issues
`get_document_context`	Read	Exact source passages by chunk ID

The audited ZSK connection allowed five read-only tools and left the three write tools unapproved. The RAG connection allowed all four retrieval tools. A batch tool's status-inspection branch can read an existing job, but the tool as a whole carries a write annotation; generic chat policy conservatively asks for approval rather than interpreting vendor-specific branches.

9.5 REST and MCP have separate roles

ZSK REST and MCP delegate to the same `Service`. RAG REST provides operator CRUD, upload/status/chunk views and search. Its MCP exposes read-only retrieval. Core's `RagController` `.validateOperation()` permits specific methods/paths/filters, not a general external HTTP proxy. Frontend never computes embeddings, queries PostgreSQL or runs the ML model. No vendor-name or machine-specific branch is required in core.

9.6 Compatibility failures are useful evidence

Recorded ZSK integration initially failed because published machine-ID regex constraints were refused by core. The external schema was adjusted to publish bounded strings while retaining server-side character checks. Date/UUID JSON serialization was corrected in the service. The later trace contains two successful read calls; earlier failure is not rewritten as success. This illustrates why transport interoperability and model-schema compatibility require actual tests.

Remote timeouts mark the profile failed with a sanitized error. A dispatched write whose result is uncertain must be reconciled; reconnecting does not authorize replay. Tool output is included as an observation, not as an instruction or raw executable script. Output-schema enforcement is limited: the SDK validates protocol envelopes, but core has no industrial schema/semantic validator for every returned field.

Engineering verdict. Independent service adapters and explicit schemas localize integration work. Schema compatibility, declared read-only behavior and result semantics must still be verified at each trust boundary.

Implementation evidence. `core/src/rag/rag.controller.ts`: `validateOperation()`; `core/src/rag/rag-client.ts`; `docs/verification/zsk-results.md`; `ui/src/components/mcp/mcp-workspace.tsx`.

CHAPTER 10

Security and Authorization

10.1 Problem and backend-enforced authority

Model proposals cannot authorize themselves. `ExecutionPolicy.prepare()` classifies a validated action using backend configuration and operation grammar. Safe approved reads may execute directly. Controlled writes need a configuration-bound per-task grant or exact approval. Dangerous or unclassified writes always need exact approval. These classes are code policy, not the LLM's assessment of a command.

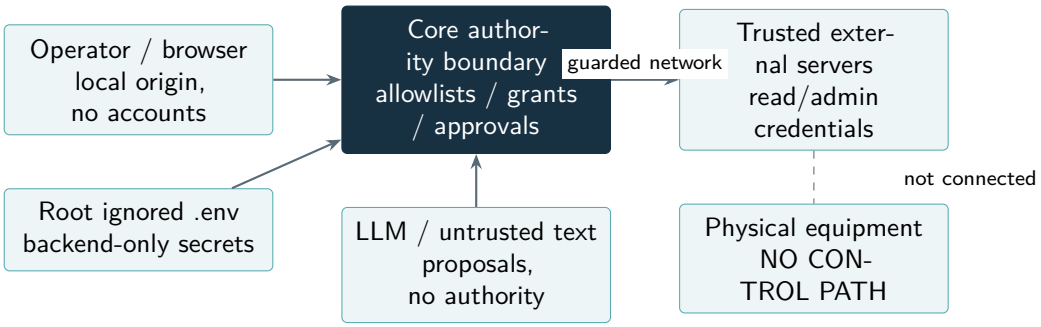


Figure 10.1: Trust boundaries and authority. Operator allowlists, backend gates and service credentials jointly constrain tool use; retrieved text cannot grant authority.

10.2 Exact chat approval and durable grants

Chat state-changing calls persist server/tool/arguments, argument/configuration hashes, ID and 15-minute expiry. `LlmService.respond()` saves phase `executing` before sending, consuming authorization. A reload/provider failure cannot automatically reuse that confirmation. Configuration/schema/permission changes invalidate its digest; decline becomes an observation and does not execute.

Durable `grant()` checks current allowlisting and operator classification in `TASK_CONTROLLED_MCP_TOOLS`; `TASK_DANGEROUS_MCP_TOOLS` takes precedence. Grants bind URL, credential-variable name, allowlist and tool descriptor. Exact task approvals bind the operation/action/policy digest and persist the decision. Unlike ordinary chat confirmations, the task-approval table has no equivalent 15-minute expiry. Do not generalize the chat expiry to every authorization mechanism.

10.3 Local exposure and secret handling

Full-stack host ports bind 127.0.0.1. `requireLocalOrigin()` allows configured local HTTP browser origins, but missing Origin is accepted; it is not user authentication. Core has no accounts/RBAC or multi-user isolation. ZSK applies separate read/admin bearer credentials; RAG authenticates its REST/MCP surface with a shared bearer key.

Root ignored `.env` holds passwords/keys. Core alone bind-mounts it for authorized settings writes; images exclude it. UI receives only four-character OpenAI prefix plus stars and saved-credential flags. Tokens entered once are saved as generated `MCP_TOKEN_<UUID>` entries, with profile references in SQL. `EnvironmentStore` serializes writes, preserves unrelated entries, fsyncs and chmods 0600. This is not a cross-process transactional secret manager.

Engineering verdict. Backend checks prevent a model from expanding its own permissions. Security is appropriate to an operator-managed local prototype, not a proven hardened multi-user industrial deployment.

Implementation evidence. `core/src/tasks/execution-policy.ts`; `core/src/config/local-origin.ts`; `core/src/config/environment-store.service.ts`; `zsk-service/zsk/auth.py`; Docker/Compose files.

10.4 Network, filesystem and industrial constraints

10.4.1 Network controls and MCP trust

`approvedUrl()` requires an exact origin from `MCP_ALLOWED_ORIGINS` or the RAG REST allowlist; credential URLs, query/fragments and redirects are rejected. `guardedFetch()` validates DNS answers during the actual socket lookup and blocks configured metadata, link-local and other unsafe ranges. Approved local private addresses are deliberately usable for Docker services. This is an origin policy with address checks, not a general network sandbox.

Read-only classification currently consumes the remote server's `readOnlyHint`. Operator trust and server-side restricted credentials are essential: a malicious server can lie. MCP explicitly treats annotations as hints from a trust boundary [10]. Core cannot prove that an arbitrary allowed "read" server has no side effects. ZSK's read credential independently refuses admin operations; this is stronger than relying on a name or annotation alone.

10.4.2 Structured local execution

Local task actions are disabled unless `TASK_WORKSPACE_ROOT` is set. `workspacePath()` resolves real paths, excludes root/outside paths, rejects protected secret/configuration directories and restricts symlinks. Workspace operations read/list/exists/write/delete have bounded content; delete requires exact approval. Writes use a temporary exclusive no-follow file, fsync and rename.

Structured commands admit only exact argument grammars for pwd, ls, cat, literal grep, bounded find and restricted git status/diff. Shell metacharacters are rejected and `spawn(...,shell:false)` uses fixed executable paths with a sanitized environment. Read/list operations often use internal filesystem implementations rather than shell utilities. There is no arbitrary SQL, Python, shell interpreter or host Docker socket. These application checks are not a complete OS sandbox; filesystem race and partition testing is limited.

10.4.3 Prompt injection and data disclosure

Model instructions treat descriptions and tool passages as untrusted, preserve source references and exclude credentials from outputs/traces. Redaction matches known saved secret strings; it cannot recognize every novel confidential value or semantic leakage. An allowed retrieval may send document passages and sensor observations to OpenAI. Local application hosting therefore does not mean fully offline processing. No general content-classification/DLP layer is implemented.

10.5 Physical maintenance boundary

Default manufacturing access should remain the five read-only tools. Append/retrain/batch persistence is software state change, not a machine-control command. There are no PLC writes, actuator tools, safety setpoint changes or autonomous maintenance authorizations. Any future equipment-control integration would require a separate industrial safety design and explicit product authorization. The operator and qualified personnel retain responsibility for deciding whether and how to intervene.

Engineering verdict. Least-privilege service credentials, exact approval and narrow command grammars materially constrain execution. Prompt instructions and metadata hints alone cannot establish safety against a compromised connected service.

Implementation evidence. `core/src/mcp/network-policy.ts`; `core/src/tasks/execution-policy.ts`: `workspacePath()`, `prepare()`, `execute()`; `core/src/llm/llm.service.ts`: `chatInstructions`.

CHAPTER 11

LLM Connection and Model Integration

11.1 Problem, design and actual provider path

The application owns a single-provider integration with OpenAI, not a broad interchangeable provider framework. `LlmService.configuration()` reads masked saved settings; `discover()` fetches `/v1/models` online using the saved key. `save()` validates model membership and thinking choice. UI offers OpenAI, API key, Model/Refresh and Thinking Instant/Medium/High. A replacement key clears the old model selection. Model listing includes specialized IDs and does not guarantee Responses, reasoning, compaction or schema compatibility.

`openAiRequest()` serializes POST requests to `/v1/responses`; chat uses `store:false`, `stream:false`, selected reasoning effort and bounded output. Instant maps to `none`. When tools exist, `tool_choice:auto` and `parallel_tool_calls:false` are supplied. Responses reasoning encrypted content is retained privately. A final completed response must contain valid output-text/refusal content; empty/incomplete results are visible failures.

11.2 How a model output becomes execution

Ordinary chat receives function calls whose provider names map back to the allowed registry tools. Core parses bounded JSON arguments, validates current schema/policy, gates writes, calls MCP and persists observations before the next inference. Durable tasks use `text.format` with a strict JSON-schema decision. That schema carries serialized JSON argument/filter/value fields; the parser then validates their contents and action semantics. Strict provider output does not replace backend validation.

Concept	Relationship to this implementation
LLM API call	Implemented HTTP communication, settings, usage and errors
Coding-specialized model	Possible selected catalog item; no automatic coding capability guarantee
External agent runtime	No Codex/Claude Code/other runtime integration in core
Owned agent loop	Implemented chat Responses loop and durable decision worker

11.3 Failures, usage and retries

401 produces a sanitized key error; 429/5xx produce availability errors and Retry-After metadata. Unsupported model/thinking/context combinations stop visibly rather than silently switch models. Durable decision usage uses returned total tokens or counted-input fallback, and supports an operator-configured conservative cost rate. Failed paid requests may lack usable usage; accounting is not an exact provider billing ledger. Chat provider failures preserve saved input/completed tool results and expose Retry response. There is no token streaming; SSE reports task progress.

Engineering verdict. The owned provider path keeps model choice and executable decisions explicit. Tight coupling simplifies local implementation but makes Responses/counting/compaction/schema compatibility a practical deployment constraint.

Implementation evidence. `core/src/llm/llm.service.ts`: `discover()`, `save()`, `respond()`, `taskDecision()`; `core/src/llm/context-window.ts`: `openAiRequest()`; `ui/src/components/llm/llm-settings.tsx`.

CHAPTER 12

Validation, Verification, and Observability

12.1 Problem: success at which layer?

A protocol-valid tool response is not a completed workflow, and a completed workflow is not a correct industrial diagnosis. The implementation has strong generic execution checks but no comprehensive equipment-diagnostic validator. These levels must remain separate in user-visible claims.

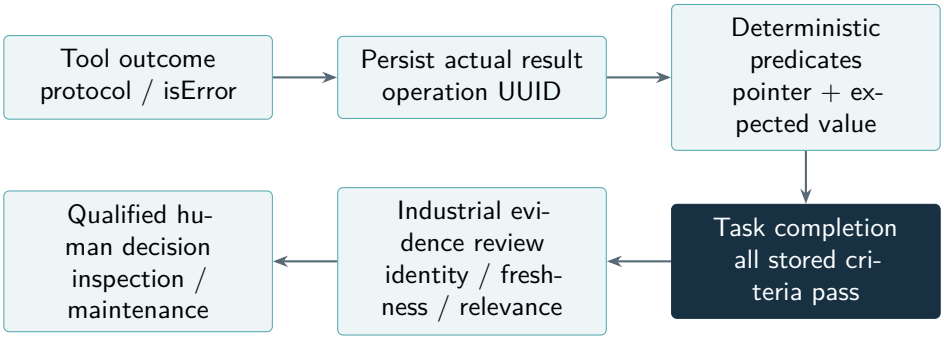


Figure 12.1: Implemented execution predicates below a separate industrial review layer. A passed task check verifies the specified stored fact, not the truth of every generated sentence.

12.2 Execution validation in code

Inputs are checked at controller/contract boundaries; MCP arguments use current discovered schema plus server-side validation. Tool errors/timeouts are persisted rather than fabricated as observations. `TasksService.verify()` runs `checkCriterion()` over SUCCEEDED operations, matching operation kind, optional server/tool and argument filters. It reads a JSON pointer and evaluates exists/equals/nonempty/gte/contains. Checks and observed operation IDs/values are recorded in `task_verifications`.

Criteria are nonempty for completion; the first committed set is retained. Reconciled operator attestations are excluded. `complete()` writes the final reply and COMPLETED state only after all criteria pass. Response-only tasks get a response-delivery criterion with `externalExecutionVerified:false`; the UI states that no external operation was verified. Pre-completion progress is the passed-criterion fraction scaled to at most 99%, not an estimate of physical maintenance completion.

12.3 Traceability and its limits

Chat traces show server/tool, duration, success and compact results/source references. Durable events record creation/planning/dispatch/result/retry/approval/checkpoint/model-output and completion; SQL operations retain actual result/error/attempts. Public task reads exclude private model context, lease owner and grants. Events/results are redacted for saved secrets; they are not tamper-evident audit records, and stored model output is not a hidden-reasoning transcript.

Task detail exposes Completion checks, Verification evidence, Tool execution history, Request and result, Events and retries. The history establishes what was observed and accepted by predicates, not a causal explanation of hardware failure.

Engineering verdict. Deterministic predicates can reject unsupported completion claims about specified external facts. Because the model proposes the initial criteria and no industrial validator is installed, passing them does not certify a maintenance recommendation.

Implementation evidence. `core/src/tasks/task-contract.ts`: `checkCriterion()`, `pointer()`, `matches()`; `core/src/tasks/tasks.service.ts`: `verify()`, `complete()`, `publicGet()`; `ui/src/components/tasks/tasks-workspace.tsx`.

PART IV — IMPLEMENTATION EVALUATION AND VERDICTS

CHAPTER 13

End-to-End System Evaluation

13.1 Evidence method and reproducible limits

This report recursively inventories first-party source, configuration, migrations, tests, plans and verification records, excluding dependency trees, secrets and generated caches. The included manifest hashes 258 readable files from the pre-authoring inventory. Primary flow analysis follows chat/task entry points through LLM, policy, MCP and external services; frontend controls were checked against source and genuine repository screenshots. External RAG and ZSK directories were accessible, including the running databases. The misspelled `CODEX_IMPLEMENTAION.md` was not found; no local copy of the requested harness paper was found, so its public text was consulted. The original supplied joblib was unavailable; initial training built the artifact from supplied code/CSV.

Read-only live checks confirmed healthy core/UI/ZSK/RAG containers, ZSK table counts and the RAG catalog/passages. A real device-scoped RAG search was executed and saved. No new paid LLM diagnosis, physical-equipment call, restart of production stacks or destructive database test was performed. Report-time npm results: core 30 passed, four optional database tests skipped; UI 22 passed, one failed. The UI regression expects exact text “Connection failed, whereas the rendered status includes a prefixed explanation. The full run therefore failed; no application fix was made in this report-only scope. Results remain distinct from historical isolated-database and service tests.

13.2 ML evaluation actually measured

Measure	Recorded result	Interpretation
Training / gap / holdout	780 / 20 / 200	Earlier training, per-machine embargo, later holdout
Holdout classes (0 / 1)	46 / 154	Synthetic faults / healthy observations
Accuracy	0.9700	194 correct of 200
Balanced accuracy	0.9653	Mean recall over both classes
Macro precision / F1	0.9518 / 0.9583	Unweighted class means
Final training rows	1,000	Deployed model uses all labeled source rows

The confusion matrix, actual rows by predicted columns in order [0,1], is $\left[\begin{smallmatrix} 44 & 2 \\ 4 & 150 \end{smallmatrix}\right]$. Thus two faults were predicted healthy and four healthy readings faulty. These are **evaluation-candidate** results from `docs/verification/zsk-final-state.json`, not an independent test of the final all-labeled forest. No industrial sensitivity, false-alarm rate per operating hour or calibrated risk is measured.

Chronological splitting and causal features avoid direct future-value leakage; the 20-reading embargo reduces neighboring labeled-window dependence. Test features can use prior measured context, consistent with inference. Same-machine recipes/fault signatures and temporal dependence remain; an embargo does not prove independence. Time-series validation assumptions require care [11]. Synthetic separability, no real maintenance labels, distribution shift and no cross-machine holdout limit generalization.

Implementation evidence. `zsk-service/zsk/zsk_model.py`: `train()`; `docs/verification/zsk-tests.log`; `docs/verification/zsk-restart.log`; included report evidence and test log.

13.3 Component evidence and verification coverage

Component	Implemented behavior / evidence	Limitation	Verdict
Manufacturing access	Live 1,000 unlabeled rows; paginated REST/MCP; source tests	Historical synthetic snapshot; two asset IDs	Implemented, identity needs resolution
ML inference	Recorded successful agent call; version/probability; batch parity in isolated tests	No industrial validation	Software proof of concept
RAG retrieval	Live 203 chunks, three publications; real scoped search	Zero curated evidence links; brochures	Grounding available, diagnosis partial
MCP	Official SDK discovery/call fixtures; eight ZSK/four RAG tools	Schema compatibility, trusted hints	Modular with explicit gates
Agent loop	Responses/tool and strict durable decision paths; fixture tests	No full ZSK maintenance run	Generic execution implemented
Context	90% compaction/delta/paging fixtures; recorded checkpoints	Lossy summaries; provider compatibility	Bounded continuity supported
Persistence	Historical real SIGKILL/SQL restart result	No distributed partition proof	Recovery supported conservatively
Security	Allowlist, approval, network and workspace tests; read-/admin credential split	No accounts or OS sandbox guarantee	Local managed boundary
Completion	Stored predicate verification; excluded attestations	Model-proposed initial criteria	External fact checks, not diagnosis
Operator output	Genuine UI/traces and cautious recorded ZSK answer	Six-part narrative not enforced	Reviewable decision support

13.4 Execution evidence and coverage limits

Historical ZSK tests ran actual forest training and 1,000 batch outputs against isolated PostgreSQL; all probabilities matched full-history inference. HTTP/MCP tests covered auth, discovery, pagination, async status and stable-key conflicts; the restart test resumed a killed batch. These validate integration, not industrial accuracy.

Core context/task tests use deterministic model fixtures. Recorded restart tests use real SQL/MCP; optional report-time SQL tests were skipped. Many RAG tests fake embeddings. No grounding, diagnosis or technician-outcome benchmark exists.

13.5 Industrial validation still required

A qualified review should verify timestamp/units/calibration, exact asset/recipe, model feature/version compatibility, passage applicability, evidence support and unresolved uncertainty. The current generic predicates can verify that a result field exists or matches a value; they cannot independently establish these industrial properties. A complete ZSK workflow was not demonstrated during report authoring. The live retrieval plus recorded ML reply are separate evidence events, and must not be combined into an invented successful agent run.

Engineering verdict. Available evidence supports implemented modular execution, synthetic inference and real retrieval. It supports no claim that the system can reliably diagnose or authorize maintenance on a physical Coperion machine.

CHAPTER 14

Design Verdicts and Limitations

Decision	Rationale / benefit	Limitation	Verdict
LLM proposes; core executes	Flexible reasoning with deterministic gates	Reasoning can be wrong	Keep authority outside text
MCP abstraction	External ML/RAG evolve independently	Hints/schemas need trust	Appropriate modular boundary
PostgreSQL checkpoints	Shared local durable state and history	Remote effects cannot transact with SQL	Recover with uncertainty states
Short-window Random Forest	Causal context and simple CPU deployment	Synthetic, no forecasting	Integration model only
Hybrid RAG	Semantic, keyword and issue retrieval routes	Extraction and corpus gaps	Inspect exact source passages
Structured decisions	Machine-readable plan/action/checks	Initial criteria can be weak	Validate again in backend
Exact approval / grants	Bind authority to action/configuration	Does not undo sent requests	Conservative for writes
Evidence predicates	Block unsupported external completion	No industrial truth oracle	Distinguish task/diagnosis
Human decision support	Operator reviews evidence and uncertainty	Requires qualified personnel	Essential product boundary

14.1 Implemented, partial and planned

Implemented: saved chat/files/settings, owned LLM/MCP loops, private compaction, durable tasks/evidence paging, schedules, write approval/reconciliation, standalone RAG ingestion/hybrid search, and versioned synthetic ZSK inference. **Partial:** industrial maintenance workflow, trusted asset mapping, complete industrial validation, calibration/freshness enforcement, comprehensive malicious-server protection and measured recommendation quality. **Planned:** operator-designed domain workflows and deterministic validators described in `docs/ROADMAP.md`. Event triggers are currently removed, not an available planned-by-assumption feature.

14.2 Prioritized development recommendations

First establish an explicit verified mapping between RAG asset UUID/`ZSK-001` and dataset `ZSK-SIM-01`; never substitute a guessed mapping. Next add approved model-input/freshness checks and a reviewed evidence/recommendation contract through external validators, keeping domain logic outside core. Curate authoritative operating/maintenance manuals with version-specific passage links; validate PDF extraction and diagnostic relevance. Collect real labeled longitudinal data under qualified review before evaluating cross-machine, recipe and fault generalization. Compare algorithms and calibrate probabilities on independent data; do not use predictions as labels.

Then test uncertain side effects under connection/partition faults, require least-privilege credentials for each server and measure operator-visible grounding/uncertainty quality. These recommendations are not implemented features or authorization to widen the project. The central answer is architectural: independent time-series and retrieval services can supply evidence to an LLM, while core persists context, validates actions and verifies specified outcomes; the human retains the industrial decision.

Implementation evidence. `docs/ROADMAP.md`; `docs/MCP_CONTRACTS.md`; `docs/TASK_EXECUTION.md`; source/verification mappings throughout Chapters 6–12.

PART V — USER MANUAL (TWO PAGES)

CHAPTER 15

Installation and Configuration

15.1 Requirements and startup

Use Docker Engine/Compose, CPU/RAM/disk for local models and PostgreSQL. Images use Node 24/Python 3.12; native npm requires Node `^22.13.0 || >=24`. No minimum RAM is specified. From the root, copy `.env.example` *only if no .env exists*, fill secrets and protect permissions. Preserve existing initialized database passwords.

```
cp .env.example .env                    # new checkout only
chmod 600 .env
docker network create meta-harness-mcp # once, if absent
docker compose -f docker-compose.yml -f docker-compose.mcp.yml up -d --build --wait
(cd rag-server && docker compose --env-file ../.env -f docker-compose.yml -f docker-compose.
    meta-harness.yml up -d --build --wait)
# New installation only: create the dedicated external volume.
docker volume create manufacturing-demo_manufacturing-data
(cd zsk-service && docker compose --env-file ../.env -f docker-compose.yml -f docker-
    compose.meta-harness.yml up -d --build --wait)
```

Required root secrets are `DB_PASSWORD`, `OPENAI_API_KEY` (or enter in UI), `RAG_DB_PASSWORD`, `RAG_API_KEY`, `MANUFACTURING_DB_PASSWORD`, `ZSK_API_KEY` and `ZSK_ADMIN_API_KEY`. Use distinct ZSK read/admin keys of at least 16 characters and a RAG API key of at least 32 characters. Use `LLM_MODEL`/`LLM_THINKING` through UI. Keep the MCP overlay on later core startup commands. If replacing the old demo, stop it *without deleting volumes* before ZSK; never run two DB containers on its volume.

15.2 Configure real UI connections

Open `http://localhost:8080`; core health is `http://localhost:8000/api/health`, RAG REST is port 8090 and ZSK port 8100. Expand **LLM**: Save API key, select an available Model/Thinking, Save settings. Unsupported combinations fail visibly.

Under **MCP > Connections/Add MCP Server**, use `http://manufacturing-ml:8000/mcp` for containerized core and bearer reference `ZSK_API_KEY`. Save, Test Connection/Discover Tools, allow the five reads and enable agent use; Save tool permissions. Under **RAG > Connection**, use MCP `http://rag-api:8090/mcp` and REST `http://rag-api:8090`; save/test and allow four read tools. Existing profiles retain credentials; new tokens are write-only entries. Operator-approved exact origins must be in root .env.

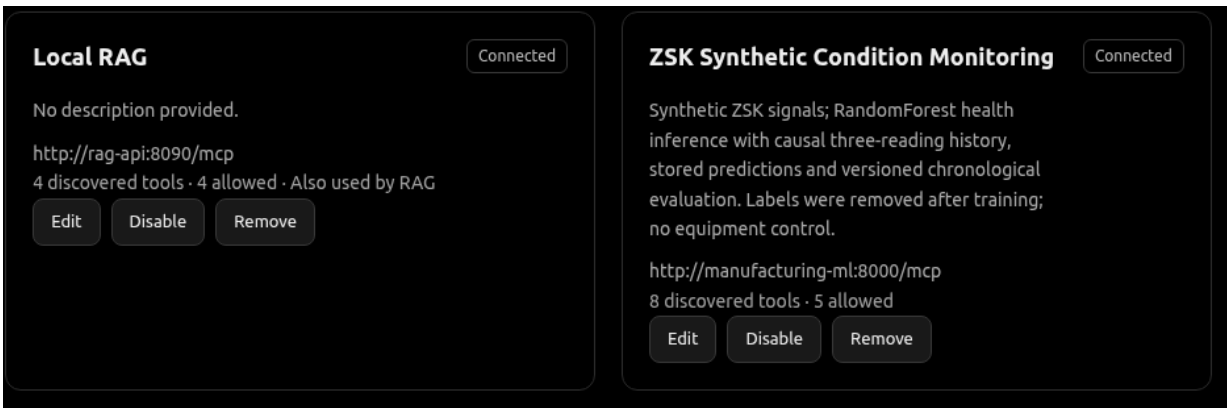


Figure 15.1: Detail from a genuine repository capture of connected RAG/ZSK profiles (9 October 2026). Saved credential metadata was hidden in the supplied publication capture.

CHAPTER 16

Operating the Decision-Support System

16.1 A short operator workflow

1. Identify the line in **RAG > Devices**; the UI has no dedicated manufacturing-line selector. State dataset code `ZSK-SIM-01` explicitly and resolve its relationship to RAG `ZSK-001` before combined analysis.
2. Ask chat to retrieve recent stored measurements and previous comparable recipe conditions. Check timestamps/units; September synthetic readings are not current October telemetry.
3. Request analysis/prediction. Review label, probability, history/warmup and model version; a Healthy label is not an operating-safety clearance.
4. Inspect supporting passages in **RAG > Knowledge Search** and document/chunk details in **Documents**. Ranking is not confidence. Chat + attachments are local files, not searchable RAG ingestion.
5. Ask for observations, model result, source evidence, interpretation, inspection priority and uncertainty separately. Treat possible causes as hypotheses.
6. Expand chat tool activity or task **Requests and outputs**; review the exact call/results. Physical inspections and maintenance decisions remain with the operator and qualified technician.

16.2 Example prompts and task recovery

"What is the current health status of ZSK-001? First verify its dataset identity and data freshness." "Analyze the gearbox vibration trend and explain possible causes using the available documentation." "Compare the latest measurements with previous operating conditions." "What inspection should the operator prioritize based on the available evidence?" Include the verified dataset code once known; do not instruct the model to guess the mapping.

Check **Run as task** before Send for durable work, or use **TASKS > Active Tasks > New task**. Monitor the live card; **View task details** opens plan, Completion checks, Verification evidence and Tool execution history/Request and result. PAUSED uses **Resume**; FAILED uses **Retry**. Resolve pending **Approve action/Reject action** and **Reconcile outcome** first. Ordinary chat uses **Confirm and run/Decline** and **Retry response**. Cancellation preserves past effects. "100% verified" means task predicates passed; response-only completion explicitly verifies no external operation. Scheduled Tasks supports cron/one-time execution; there is no trigger control.

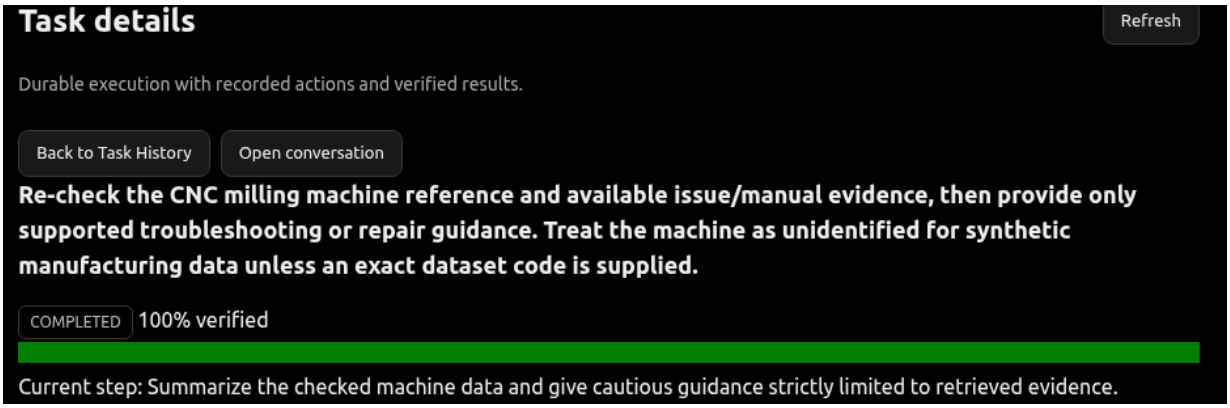


Figure 16.1: Detail from a genuine task-detail capture. This recorded CNC example illustrates the actual controls, not a ZSK diagnosis; its verified predicates do not certify industrial correctness.

Implementation evidence. `ui/src/components/tasks/tasks-workspace.tsx`; `ui/src/components/chat/task-feedback.tsx`; `ui/src/components/rag/management.tsx`; `ui/src/App.tsx`; service READMEs and `docs/OPERATIONS.md`.

References

[1] Coperion. Condition monitoring on zsk extruder gearbox increases overall machine efficiency. Plastics in Focus 02/2022. https://www.coperion.com/en/news-media/newsletter/2022/plastics-in-focus-edition-022022/condition-monitoring-on-zsk-extruder-gearbox, 2022.

[2] Andrew K. S. Jardine, Daming Lin, and Dragan Banjevic. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510, 2006.

[3] scikit-learn developers. Randomforestclassifier api documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html, 2026. Consulted for ensemble/probability semantics; repository locks own runtime version.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. https://arxiv.org/abs/2005.11401.

[5] Coperion. Zsk and sts twin screw extruders: More efficiency in compounding and recycling. September press release, pp. 1–6. https://www.coperion.com/media/tnqbsvnq/coperion_k_2025_zsk_sts_twin_screw_extruders_en.pdf, 2025. Live RAG local title: ZSK Condition Monitoring Extruder Brochure; passages on p. 3.

[6] Coperion. Zsk twin screw extruders. Brochure B-600304-en, August. https://www.coperion.com/media/kdxpvjto/b-600304-en-zsk_2025-08.pdf, 2025. Live RAG publication; relevant condition-monitoring passage p. 28.

[7] Coperion. Extruder modernization and service. https://www.coperion.com/media/8959/coperion_service_extruder_modernization.pdf. Undated publication in live RAG; p. 4 discusses monitoring bottlenecks; not a diagnostic manual.

[8] Paul Barbaste, Tristan Darrigol, Germain Vu, and Tom Wiltberger. Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems. arXiv:2609.00006v1, 2026. Sections 2, 6, 9 and 10 consulted; accessed 9 October 2026.

[9] Model Context Protocol. Tools: Specification revision 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/server/tools, 2025. Accessed 9 October 2026.

[10] Ola Hungerford, Sam Morrow, and Luca Chang. Tool annotations as risk vocabulary: What hints can and can't do. MCP Blog, 16 March. https://blog.modelcontextprotocol.io/posts/2026-03-16-tool-annotations/, 2026. Accessed 9 October 2026.

[11] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.